

Unidad 4:

Objetos definidos por el usuario (parte 2)

Desarrollo Web en Entorno Cliente

Encapsulamiento

Encapsulamiento

El encapsulamiento en JavaScript es un concepto fundamental en la programación orientada a objetos que permite **restringir el acceso** a ciertas partes de un objeto y ocultar su estado interno.

Ventajas

- **Control de Acceso:** Permite definir qué partes del objeto son accesibles y cuáles no.
- **Mantenimiento:** Facilita el mantenimiento del código al ocultar la complejidad interna.
- **Reutilización:** Promueve la reutilización del código, ya que puedes compartir módulos o clases sin exponer su implementación.

Desventajas

- **Sobrecarga:** Puede introducir una pequeña sobrecarga en términos de rendimiento debido a la necesidad de métodos getter/setter.
- **Complejidad:** Puede hacer que el código sea más complejo y menos intuitivo, especialmente para principiantes.

Encapsulamiento

Desde la introducción de las Clases en ES6 para el manejo de los objetos, encapsulamiento se puede conseguir declarando propiedades y métodos privados haciendo uso del carácter “#”.

```
class NombreClase {  
    #propiedad;  
    constructor(valor) {  
        this.#propiedad = valor;  
    }  
    get propiedad() {  
        return this.#propiedad;  
    }  
    set propiedad(prop) {  
        this.#propiedad = prop;  
    }  
    #metodo() {  
        alert("Esto es un método  
        privado");  
    }  
    metodo2() {  
        this.#metodo();  
    }  
}
```

Módulos ES6

Módulos ES6

Definición: Un módulo es un archivo JavaScript que puede exportar e importar funcionalidades.

Ventajas:

- Encapsulamiento del código.
- Reutilización de componentes.
- Mejora de la mantenibilidad del proyecto.

Módulos ES6

Para poder utilizar los módulos ES6 en tu proyecto es necesario que cuando especifiques en el HTML los archivos .js indiquemos que se trata de tipo módulo.

```
<script type="module" src="./ruta.js"></script>
```

Es muy importante que la ruta comience por `./` y que se especifique que el formato del archivo es `.js`

Módulos ES6

Exportación nombrada:

Se trata de un método de exportación de funcionalidades en el que se pueden exportar múltiples elementos desde un mismo módulo; pero se deben importar usando el mismo nombre con el que fueron exportados.

```
// archivo.js
export const PI = 3.14;
export function suma(a, b) {
  return a + b;
}
```

Módulos ES6

Importación nombrada:

Se importa usando llaves {} y el nombre exacto de la exportación.

Tiene la ventaja de que puedes importar solo las exportaciones que necesites y renombrarlas si es necesario.

```
import { suma as add } from './archivo.js';
```

Módulos ES6

Exportación por defecto:

Solo se puede tener una exportación por defecto por módulo. Puede ser cualquier tipo de valor (objeto, función, clase, etc.).

```
// archivo.js  
const multiplicar = (a, b) => a * b;  
export default multiplicar;
```

Módulos ES6

Importación por defecto:

Se importa sin llaves {} y puede tener cualquier nombre al importarla.

Es útil cuando deseas exportar un solo valor o funcionalidad principal del módulo.

```
import multiplicar from './archivo.js';
```

Módulos ES6

Buenas Prácticas

- **Usa nombres descriptivos:** Elige nombres claros para funciones y variables que reflejen su propósito.
- **Exporta solo lo necesario:** Limita las exportaciones a lo esencial para mantener el módulo limpio y enfocado.
- **Organiza tus módulos:** Agrupa funciones relacionadas en módulos específicos para mejorar la mantenibilidad.
- **Evita dependencias circulares:** Mantén la estructura simple para prevenir errores complejos relacionados con importaciones.
- **Documenta tu código:** Añade comentarios y documentación para explicar la funcionalidad y facilitar su comprensión.